

TU BCA File Handling Questions with Solutions

Prepared by: Er.Arjun Neupane

August 18, 2025

Question 1 Differentiate between text files and binary files. Suppose a file "num.txt" contains integers. Write a program to extract prime numbers and write them to "Prime.txt".

Differences

Text Files	Binary Files
Human-readable	Machine-readable
Store data as plain text	Store data in binary format
Slower I/O operations	Faster I/O operations
Larger file size	Smaller file size
Platform dependent newlines	No newline conversion

Program Solution

```
1 #include <stdio.h>
2
3 // Function to check prime number
4 int isPrime(int n) {
5     if (n <= 1) return 0;
6     for (int i = 2; i * i <= n; i++)
7         if (n % i == 0) return 0;
8     return 1;
9 }
10
11 int main() {
12     FILE *fin = fopen("num.txt", "r"); // Open input file in read mode
13     FILE *fout = fopen("Prime.txt", "w"); // Open output file in write
mode
14
15     if (fin == NULL || fout == NULL) {
16         printf("Error opening files!");
17         return 1;
18     }
19
20     int num;
```

```

21     while (fscanf(fin, "%d", &num) != EOF)    // Read until end of file
22         if (isPrime(num))
23             fprintf(fout, "%d\n", num);    // Write prime numbers
24
25     fclose(fin);    // Close input file
26     fclose(fout);    // Close output file
27     return 0;
28 }

```

Listing 1: Prime Number Extraction

Step Explanation

1. **File Opening:** The program opens "num.txt" in read mode and "Prime.txt" in write mode.
2. **Prime Check:** For each number read from the input file, the `isPrime()` function checks if it's prime.
3. **Writing Output:** Prime numbers are written to the output file using `fprintf()`.
4. **File Closing:** Both files are properly closed to ensure data integrity.
5. **Error Handling:** Checks if files opened successfully.

Question 2 Define file opening modes. Write a program to delete a specific student record (by roll number) from "students.dat".

File Opening Modes

Mode	Description
"r"	Read mode (file must exist)
"w"	Write mode (creates/truncates)
"a"	Append mode (creates if doesn't exist)
"r+"	Read/write (file must exist)
"w+"	Read/write (creates/truncates)
"a+"	Read/append (creates if doesn't exist)
"rb"	Read binary mode
"wb"	Write binary mode

Program Solution

```

1  #include <stdio.h>
2
3  struct Student {
4      int roll;
5      char name[30];
6  };
7

```

```

8 int main() {
9     FILE *fp = fopen("students.dat", "rb"); // Open in binary read
mode
10     if (fp == NULL) {
11         printf("Error opening file!");
12         return 1;
13     }
14
15     int del_roll;
16     printf("Enter roll to delete: ");
17     scanf("%d", &del_roll);
18
19     FILE *temp = fopen("temp.dat", "wb"); // Temporary file
20     struct Student s;
21     int found = 0;
22
23     while (fread(&s, sizeof(s), 1, fp)) {
24         if (s.roll != del_roll) {
25             fwrite(&s, sizeof(s), 1, temp); // Write all except
matching
26         } else {
27             found = 1;
28         }
29     }
30
31     fclose(fp);
32     fclose(temp);
33
34     if (!found) {
35         printf("Record not found!\n");
36         remove("temp.dat"); // Delete temporary file
37     } else {
38         remove("students.dat"); // Delete original
39         rename("temp.dat", "students.dat"); // Rename temp to original
40         printf("Record deleted successfully!\n");
41     }
42
43     return 0;
44 }

```

Listing 2: Record Deletion Program

Step Explanation

1. **File Opening:** Original file opened in binary read mode, temporary file in write mode.
2. **Record Filtering:** Reads each record and only writes non-matching records to temp file.
3. **File Management:** After processing, original file is deleted and temp file is re-named.
4. **Error Handling:** Checks if record exists and handles file operations safely.

Question 3 Write a program to accept 100 numbers from the user and store odd numbers in "odd.txt" and even numbers in "even.txt". Then display contents of "odd.txt".

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *fodd = fopen("odd.txt", "w");
5     FILE *feven = fopen("even.txt", "w");
6
7     if (fodd == NULL || feven == NULL) {
8         printf("Error creating files!");
9         return 1;
10    }
11
12    printf("Enter 100 numbers:\n");
13    for (int i = 0; i < 100; i++) {
14        int num;
15        scanf("%d", &num);
16        if (num % 2 == 0) {
17            fprintf(feven, "%d\n", num); // Write even numbers
18        } else {
19            fprintf(fodd, "%d\n", num); // Write odd numbers
20        }
21    }
22
23    fclose(fodd);
24    fclose(feven);
25
26    // Display odd numbers
27    printf("\nOdd numbers:\n");
28    fodd = fopen("odd.txt", "r");
29    char line[10];
30    while (fgets(line, sizeof(line), fodd)) {
31        printf("%s", line);
32    }
33    fclose(fodd);
34
35    return 0;
36 }
```

Listing 3: Odd-Even Number Separator

Part 3.1 Key Features

- Uses ternary operator for compact number classification
- Demonstrates both writing and reading file operations
- Shows sequential file processing
- Includes proper file closing

Question 4 Explain file positioning functions. Write a program to reverse the contents of a file "data.txt" and save it in "reverse.txt".

Part 4.1 File Positioning Functions

Function	Description
fseek()	Moves file pointer to specified position
ftell()	Returns current file pointer position
rewind()	Sets file pointer to beginning of file
fgetpos()	Gets current file position
fsetpos()	Sets file position

Part 4.2 Program Solution

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     FILE *fin = fopen("data.txt", "r");
6     if (fin == NULL) {
7         printf("Error opening input file!");
8         return 1;
9     }
10
11     // Move to end to find file size
12     fseek(fin, 0, SEEK_END);
13     long len = ftell(fin);
14
15     // Allocate buffer for entire file
16     char *buf = malloc(len + 1);
17     if (buf == NULL) {
18         printf("Memory allocation failed!");
19         fclose(fin);
20         return 1;
21     }
22
23     // Read entire file into buffer
24     fseek(fin, 0, SEEK_SET);
25     fread(buf, 1, len, fin);
26     buf[len] = '\0'; // Null-terminate
27
28     FILE *fout = fopen("reverse.txt", "w");
29     if (fout == NULL) {
30         printf("Error creating output file!");
31         free(buf);
32         fclose(fin);
33         return 1;
34     }
35
36     // Write in reverse order
37     for (long i = len - 1; i >= 0; i--) {
38         fputc(buf[i], fout);
```

```

39     }
40
41     // Cleanup
42     free(buf);
43     fclose(fin);
44     fclose(fout);
45
46     printf("File reversed successfully!\n");
47     return 0;
48 }

```

Listing 4: File Reversal Program

Part 4.3 Step Explanation

1. **File Size Determination:** Uses `fseek()` and `ftell()` to get file length
2. **Memory Allocation:** Dynamically allocates buffer to hold entire file
3. **Bulk Reading:** Reads complete file in one operation for efficiency
4. **Reverse Writing:** Writes characters in reverse order using `fputc()`
5. **Memory Management:** Properly frees allocated memory and closes files

Question 5 Create a structure "Employee" (ID, name, salary). Write a program to update salaries by 10% for employees with salary ; 25,000 in "emp.dat".

Program Solution

```

1  #include <stdio.h>
2
3  struct Employee {
4      int id;
5      char name[30];
6      float salary;
7  };
8
9  int main() {
10     FILE *fp = fopen("emp.dat", "rb+");
11     if (fp == NULL) {
12         printf("Error opening employee database!");
13         return 1;
14     }
15
16     struct Employee e;
17     int count = 0;
18
19     while (fread(&e, sizeof(e), 1, fp) == 1) {
20         if (e.salary < 25000) {

```

```

21 // Store current position
22 long pos = ftell(fp) - sizeof(e);
23
24 // Update salary
25 e.salary *= 1.10;
26
27 // Move back and update record
28 fseek(fp, pos, SEEK_SET);
29 fwrite(&e, sizeof(e), 1, fp);
30
31 // Return to next position
32 fseek(fp, 0, SEEK_CUR);
33 count++;
34 }
35 }
36
37 fclose(fp);
38 printf("%d records updated successfully!\n", count);
39 return 0;
40 }

```

Listing 5: Salary Update Program

Step

- **Random Access:** Uses `fseek()` for in-place updates
- **Binary File Handling:** Reads/writes structure objects directly
- **File Position Management:** Maintains proper file pointer position
- **Selective Update:** Only modifies qualifying records

Question 6 Differentiate between `fprintf()` and `fwrite()`. Write a program to store cricket player records (name, runs) in a binary file "players.dat" and display players with runs > 1000.

Differences: `fprintf()` vs `fwrite()`

<code>fprintf()</code>	<code>fwrite()</code>
Text formatting	Binary data writing
Human-readable output	Machine-efficient storage
Slower for large data	Faster for bulk data
Formatted conversion	Direct memory dump
Uses format specifiers	Uses byte count

Program Solution

```
1 #include <stdio.h>
2 #include <string.h>
3
4 struct Player {
5     char name[30];
6     int runs;
7 };
8
9 void addPlayers() {
10     struct Player team[] = {
11         {"Kohli", 12000},
12         {"Rohit", 9500},
13         {"Dhoni", 10500},
14         {"Pant", 800},
15         {"Bumrah", 150}
16     };
17
18     FILE *fp = fopen("players.dat", "wb");
19     if (fp == NULL) {
20         printf("Error creating file!");
21         return;
22     }
23
24     fwrite(team, sizeof(struct Player), 5, fp);
25     fclose(fp);
26 }
27
28 void displayTopPerformers() {
29     FILE *fp = fopen("players.dat", "rb");
30     if (fp == NULL) {
31         printf("Error opening file!");
32         return;
33     }
34
35     printf("Top Performers (1000+ runs):\n");
36     printf("-----\n");
37
38     struct Player p;
39     while (fread(&p, sizeof(p), 1, fp) == 1) {
40         if (p.runs > 1000) {
41             printf("%-20s %6d\n", p.name, p.runs);
42         }
43     }
44
45     fclose(fp);
46 }
47
48 int main() {
49     addPlayers();           // Create initial data
50     displayTopPerformers(); // Show filtered results
51     return 0;
52 }
```

Listing 6: Cricket Records Program

steps

1. **Binary File Creation:** Demonstrates writing array of structures
2. **Data Filtering:** Reads file and displays only qualifying records
3. **Modular Design:** Separate functions for different operations
4. **Formatted Output:** Uses field width for aligned display